

Secure Coding Review Report

CodeAlpha Cyber Security Internship – Task 3

Project Title: Secure Coding Review – Python Login System

Programming Language: Python

Review Method: Manual Code Inspection

Author: Anushree Anil

1. Introduction

Secure coding is the practice of writing software that protects applications from security threats and vulnerabilities. Even small coding mistakes can lead to data breaches, unauthorized access, and financial losses.

This review evaluates a simple Python login application to identify common security flaws. The objective is to analyze the vulnerable implementation, explain the associated risks, and recommend secure coding techniques to improve the application's security.

2. Application Overview

The application is a basic Python login system that accepts a username and password and checks the credentials against a SQLite database.

Two versions of the application are included:

- Vulnerable Login System
- Secure Login System

The vulnerable version intentionally contains security flaws for educational purposes, while the secure version demonstrates recommended coding practices.

3. Code Review Methodology

The security review was conducted using **manual code inspection**.

The following areas were examined:

- User Input Handling
- Database Queries
- Password Management
- Exception Handling
- Authentication Logic
- Error Messages

- Input Validation

Each identified issue was analyzed based on its potential security impact and assigned a severity level.

4. Security Vulnerabilities Identified

Vulnerability 1: SQL Injection

Severity: High

Description

The application builds SQL queries by directly concatenating user input into the SQL statement.

Risk

Attackers can manipulate SQL queries to bypass authentication or access sensitive database information.

Recommendation

Use parameterized SQL queries (prepared statements) instead of string concatenation.

Vulnerability 2: Hardcoded Credentials

Severity: High

Description

Administrative credentials are stored directly inside the source code.

Risk

Anyone with access to the source code can view sensitive credentials.

Recommendation

Store credentials securely using environment variables or secure configuration files.

Vulnerability 3: Plain Text Password Handling

Severity: High

Description

Passwords are compared as plain text instead of using encrypted hashes.

Risk

If the database is compromised, user passwords can be exposed immediately.

Recommendation

Hash passwords using secure algorithms before storing and comparing them.

Vulnerability 4: Information Disclosure

Severity: Medium

Description

Detailed database error messages are displayed directly to the user.

Risk

Attackers may gain information about the application's internal structure.

Recommendation

Display generic error messages while logging technical details internally.

Vulnerability 5: Missing Input Validation

Severity: Medium

Description

The application accepts user input without validation.

Risk

Invalid or malicious input may cause unexpected behavior.

Recommendation

Validate and sanitize all user inputs before processing them.

5. Secure Coding Improvements

The secure implementation includes several improvements:

- Parameterized SQL queries to prevent SQL Injection.
- Password hashing using SHA-256.
- Input validation to reject empty values.
- Generic error messages to avoid information leakage.
- Proper exception handling.
- Secure database connection management.

These improvements significantly reduce the application's exposure to common security threats.

6. Comparison

Vulnerable Version	Secure Version
--------------------	----------------

SQL Injection possible	Parameterized Queries
------------------------	-----------------------

Hardcoded credentials	No hardcoded secrets
-----------------------	----------------------

Plain text passwords	SHA-256 hashed passwords
----------------------	--------------------------

Vulnerable Version	Secure Version
Detailed errors shown	Generic error messages
No validation	Input validation implemented

7. Secure Coding Best Practices

The following practices are recommended for secure software development:

- Validate all user input.
 - Use parameterized SQL queries.
 - Never hardcode passwords or API keys.
 - Hash passwords before storage.
 - Implement proper exception handling.
 - Follow the Principle of Least Privilege.
 - Keep dependencies updated.
 - Perform regular security testing and code reviews.
 - Log security events without exposing sensitive information.
 - Follow secure coding standards throughout development.
-

8. Conclusion

The manual review successfully identified several common security vulnerabilities in the vulnerable login application. The secure implementation demonstrates how relatively small code changes can significantly improve application security. Secure coding practices help protect applications against common cyber threats, reduce the likelihood of security breaches, and improve software reliability. Regular code reviews, security testing, and adherence to secure development guidelines are essential components of modern software development.

Overall Result

Application Reviewed: Python Login System

Review Method: Manual Code Inspection

Vulnerabilities Found: 5

High Severity: 3

Medium Severity: 2

Low Severity: 0

Status: Remediation Successfully Demonstrated